



第09章 指针及动态内存管理

张仁斌@合肥工业大学软件学院

本章主要内容



9.1 指针与指针变量



9.2 指针运算

9.3 用指针处理数组

9.4 指针数组与数组指针

9.5 指针作为函数参数

9.6 指针型函数和指向函数的指针

9.7 动态内存分配

9.8 作业与实践



引言

◆数据在内存中以**字节**为基本单位进行存储

■变量定义：在内存中分配空间（内存单元）

■变量赋值：将值的编码存储在分配的内存单元

◆变量在哪——取变量的地址

◆变量地址

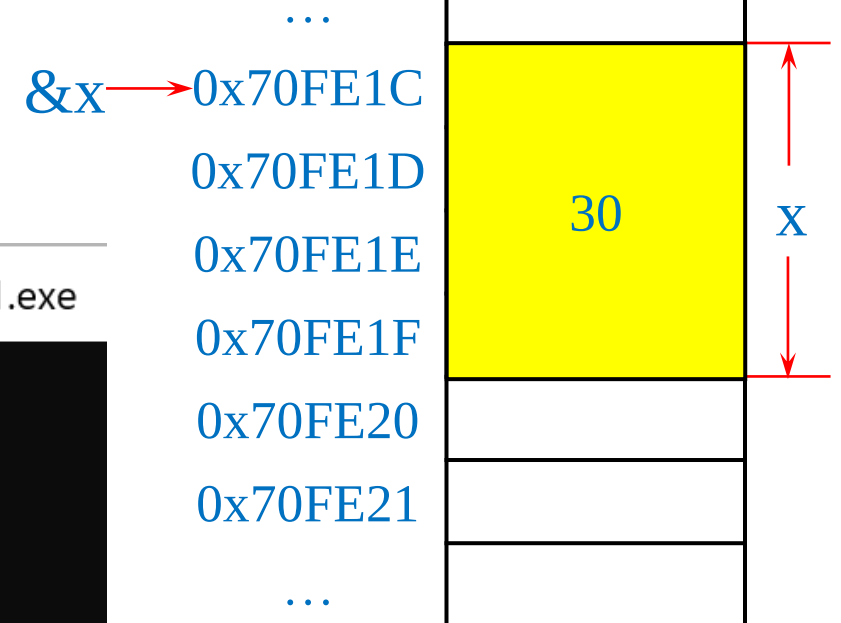
■变量在内存中占用的存储单元的首地址

```
cout << x << endl;
cout << sizeof(x) << endl;
cout << &x << endl;
cout << sizeof(&x) << endl;
```

```
>_ D:\Edu-CPP\chap09\ex09-01.exe
30
4
0x70fe1c
8
```

`int x;`

`x = 30;` 地址 内存(RAM)



■什么是**指针**——地址

■什么是**指针变量**——用于存放内存**地址**的变量



指针变量的定义

◆ 指针类型与指针变量

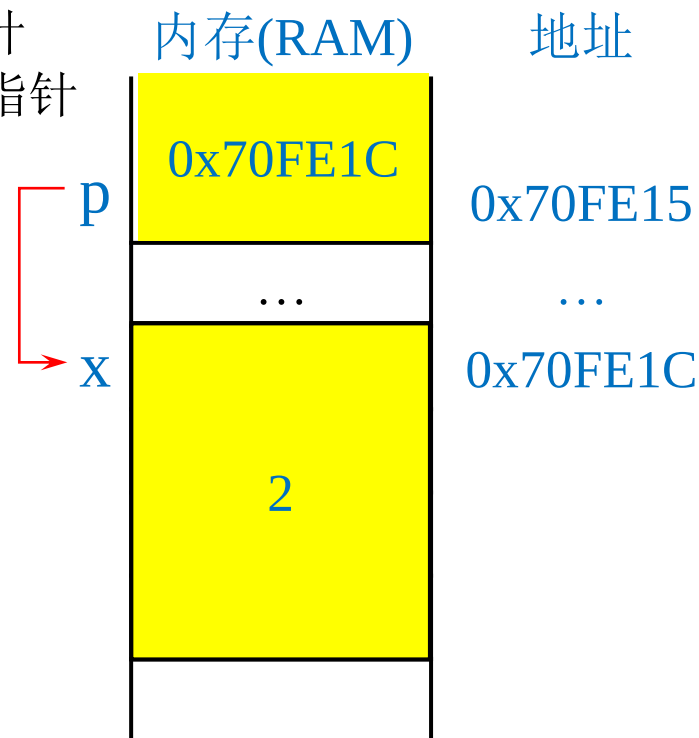
`int a, *pi;` // `pi`是指针变量，其类型是`int *`，即`pi`是指向整数的指针
`float *p;` // `p`是指针变量，其类型是`float *`，即`p`是指向浮点数的指针

也可以写为：
`float* p;`
或
`float * p;`

`int x = 3;`
`int *p;`
`p = &x;`
`*p = 2;`

将变量地址存入指针变量后，该指针指向该变量

指针必须指向对象后才能引用



◆ 根据变量的使用属性，变量分为：

■ 值变量

➢ 在其存储空间中存放的是该变量的值，用变量名存取变量值

■ 地址变量（指针变量）

➢ 变量专用于存放另一个变量的地址（指针）

◆ 变量中数据的存取方法

■ 直接利用变量名存取数据

■ 利用指针变量存取相应变量中的数据

`int *p;`等价于`int* p;`等价于`int * p;`
`int* p1, p2;`等价于`int *p1, p2;`



◆与地址相关的两种运算

■取地址运算（&）：&a表示取变量a地址

■取值运算（*）：*p表示取指针p指向变量的值

◆&和*为互补运算

```
int x, *p;
```

```
p = &x;
```

■则

&*p等同p

*&x等同x

p
*p
&p
sizeof(p)
sizeof(*p)



```
int x = 30;
```

```
int *p; // *是类型标记
```

```
p = &x;
```

```
*p = 60; // *是运算符
```

```
cout << p << " " << &x << " " << &*p << "\n";  
cout << *p << " " << x << " " << *&x;
```

```
D:\Edu-CPP\chap09\ex09-02.exe
```

```
0x70fe04 0x70fe04 0x70fe04  
60 60 60
```



指针变量的初始化与赋值

◆ 指针变量的初始化

```
int a, *pi = NULL, *pi2 = &a;
```

```
int *q = pi2;
```

```
double *pa = &a; ❌
```

◆ 指针变量的赋值

```
int a = 50, b[10], *p, *q, *r;
```

```
p = &a; // p指向a
```

```
q = b; // q指向数组b
```

```
*p = 2; // 将a的值改为2
```

```
*r = 2; ❌
```

使用指针前，必须
赋一个合法的值

注意事项

- 必须用地址为指针初始化，不能使用非法指针访问内存
- 用变量地址作为初值时，变量类型应与指针类型一致，指针指向的内存中存放的数据类型与指针类型必须相符
- 指针的类型是它所指向变量的类型，不是指针本身存储数据值类型，任何指针存储的地址都是**unsigned long int型**

`sizeof(int *) == sizeof(float *) == sizeof(char *) == sizeof(char **)`

- 可以用一个已赋初值的指针去初始化另一个指针变量
- 给指针变量赋的值必须是**地址常量**、**地址变量**或整数0（**NULL**，表示**空指针**），不允许赋其它值
- 允许声明指向void类型指针（`void *p;`），该指针可以被赋予任何类型变量的地址



```
int a = 10, b=10, *ptr1, *ptr2;
ptr1 = &a;
ptr2 = &b;
cout << (*ptr1 == *ptr2) << endl;
cout << (ptr1 == ptr2) << endl;

*ptr1 = 20;
b = 15;
cout << a << endl;
cout << *ptr2 << endl;

int x, *ptr3 = &x, *ptr4 = &x;
cout << ptr3 << " " << ptr4 << endl;
cout << &ptr3 << " " << &ptr4 << endl;
```

D:\Edu-CPP\chap09\ex09-03.exe

```
1
0
20
15
0x70fdf4 0x70fdf4
0x70fde8 0x70fde0
```

	内存(RAM)	地址
ptr4	0x70FDF4	0x70FDE0
ptr3	0x70FDF4	0x70FDE8
	...	...
x		0x70FDF4



几种特殊指针

◆ **常量指针**：指向常量的指针，**不能利用指针**改变其指向的对象

■ 阉割指针操作内存空间的能力

```
int a = 0, b = 0;  
const int *p = &a;  
p = &b;  
*p = 1; ✗  
a = b = 1;
```

◆ **指针常量**：指针指向的位置不能改变

■ 阉割指针移动的能力

```
int a = 0, b = 0;  
int * const p = &a;  
p = &b; ✗  
*p = 1;
```

◆ **指向常量的常指针**：指针指向的位置不能修改，指针指向的值也不能利用指针进行修改

```
int a = 0, b = 0;  
const int const *p = &a;  
p = &b; ✗  
*p = 1; ✗
```



◆ 泛指指针void

- 单向兼容其他类型指针，即可将其他类型的指针变量赋给void指针变量，反过来则必须进行**强制类型转换**
- void指针在没有赋值前不能作为右值被引用

```
void *p1 = NULL;  
int *p2 = NULL, a;  
p1 = &a;  
p2 = (int *)p1;
```

本章主要内容



9.1 指针与指针变量

9.2 指针运算



9.3 用指针处理数组

9.4 指针数组与数组指针

9.5 指针作为函数参数

9.6 指针型函数和指向函数的指针

9.7 动态内存分配

9.8 作业与实践

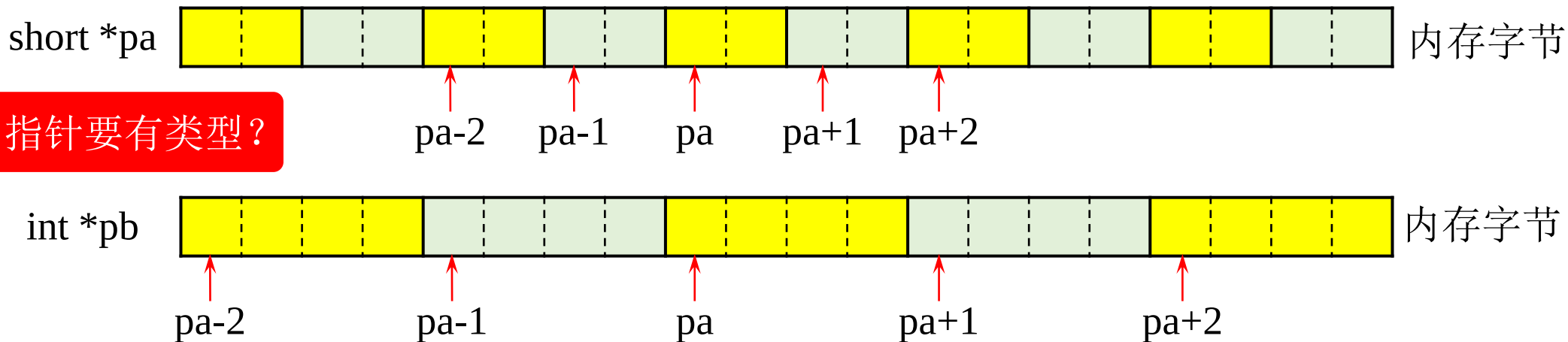
算术运算



◆ 指针变量存放变量的地址，因此指针的运算一定要符合地址逻辑

■ 指针与整型值加减，表示指针下移或上移内存单元（数据）个数之后的内存地址，内存单元大小与指针类型相关

➤ 例如：指针p和整数n的算术运算 $p \pm n$ 后p指向的新地址是 $p \pm \text{sizeof}(p\text{的类型}) * n$



■ 指针与指针的减运算要求相减的两个指针属于同一类型，其结果表示两个指针之间的数据的个数

关系运算



- ◆ 类型相同的两个指针进行关系运算，用以确定它们所指向数据的前后顺序

$p1 == p2$

$p2 > p1$

$p2 < p1$

- ◆ 当两个指针指向同一数组时，如果指针的值不同，那么取值大的指针所指的元素位于取值小的指针所指元素后



赋值运算

- ◆ 把一个变量的地址赋予一个指向相同数据类型的指针

```
char a, *p;
```

```
p = &a;
```

- ◆ 把一个指针的值赋予相同数据类型的另一个指针

```
int *p, *q, a;
```

```
q = &a;
```

```
p = q;
```

- ◆ 把数组的地址赋予指向相同数据类型的指针

```
char a[10], *pa;
```

```
pa = a;
```

本章主要内容



9.1 指针与指针变量

9.2 指针运算

9.3 用指针处理数组



9.4 指针数组与数组指针

9.5 指针作为函数参数

9.6 指针型函数和指向函数的指针

9.7 动态内存分配

9.8 作业与实践

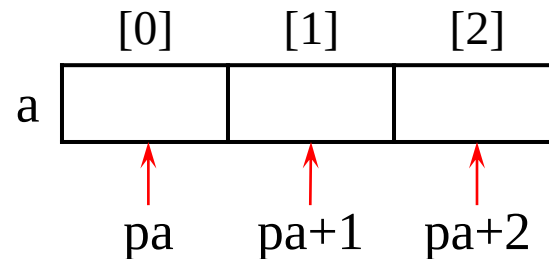


利用指针访问一维数组元素

- ◆ 数组名实际上是一个指针（**常量**，数组空间分配后不能更改）

```
int a[3];
```

```
int *pa = a; // 等价于: int *pa = &a[0];
```



- ◆ 引用数组元素两种等价方法

下标法: `a[i]`, `pa[i]`

指针法: `*(a+i)`, `*(pa+i)`

■ 例如:

`pa[0]`, `a[0]`, `*(a)`, `*pa` 值相互等价
`&pa[0]`, `&a[0]`, `a`, `pa` 指针相互等价

`pa[1]`, `a[1]`, `*(a+1)`, `*(pa+1)`
`&pa[1]`, `&a[1]`, `(a+1)`, `(pa+1)`

`pa[2]`, `a[2]`, `*(a+2)`, `*(pa+2)`
`&pa[2]`, `&a[2]`, `(a+2)`, `(pa+2)`

`a++`; ✗
`pa++`;

```
cout << sizeof(a); // 输出12  
cout << sizeof(pa); // 输出8
```



◆ 访问一维数组元素示例（以下几种方法都是等价的）

非字符串数组（以下几种方法等价）

```
int a[5] = {1, 2, 3, 4, 5}, i, *pa = a;
```

```
for(i = 0; i < 5; i++)
```

```
    cout << a[i] << '\t';
```

```
cout << endl;
```

```
for(i = 0; i < 5; i++)
```

```
    cout << *(a+i) << '\t';
```

```
cout << endl;
```

```
for(i = 0; i < 5; i++)
```

```
    cout << pa[i] << '\t';
```

```
cout << endl;
```

```
for(i = 0; i < 5; i++)
```

```
    cout << *pa++ << '\t';
```

```
cout << endl;
```

```
for(pa = a; pa < a+5; pa++)
```

```
    cout << *pa << '\t';
```

*(a + 1)不同于*a + 1

思考其运算顺序

字符串方式一

```
char s[10] = "abcdef", *p = s;
```

```
    for(; *p != '\0'; p++) .....
```

字符串方式二（常字符串，不建议）

```
char *p = "abcdef"; // 编译警告
```

或：

```
char *p;
```

```
p = "abcdef"; // 编译警告
```

.....

D:\Edu-CPP\chap09\ex09-04.exe

1	2	3	4	5
1	2	3	4	5
1	2	3	4	5
1	2	3	4	5
1	2	3	4	5



利用指针访问二维数组元素

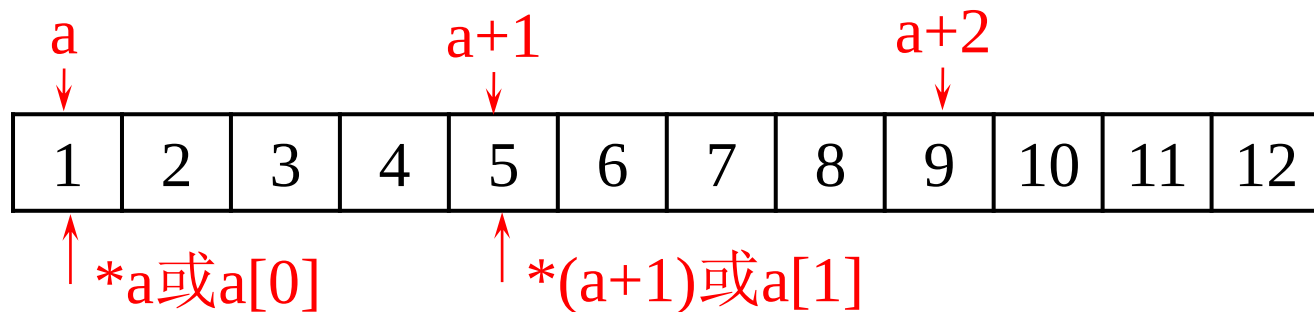
◆把二维数组看作是一种特殊的一维数组：它的元素又是一个一维数组

```
int a[3][4]={ {1,2,3,4}, {5,6,7,8}, {9,10,11,12}};
```

a $\left\{ \begin{array}{l} a[0] \text{ ---} a[0][0] \ a[0][1] \ a[0][2] \ a[0][3] \\ a[1] \text{ ---} a[1][0] \ a[1][1] \ a[1][2] \ a[1][3] \\ a[2] \text{ ---} a[2][0] \ a[2][1] \ a[2][2] \ a[2][3] \end{array} \right.$

$\rightarrow *(a+i)+j$ 或 $a[i]+j$

	1	2	3	4
$a+i$	5	6	7	8
	9	10	11	12



① $a+i$ 地址类型为**行地址**，地址增加 $i*\text{sizeof}(\text{int})*4$

② $*(a+i)$ 值不变，地址类型变为**列地址**（也可写作 $a[i]$ ）， $*(a+i)+j$ 地址增加 $j*\text{sizeof}(\text{int})$



◆ 数组行的起始地址

■ 第 0 行的首地址为 a

➤ a 、 $\&a[0][0]$ 、 $a[0]$ 、 $*(a+0)$

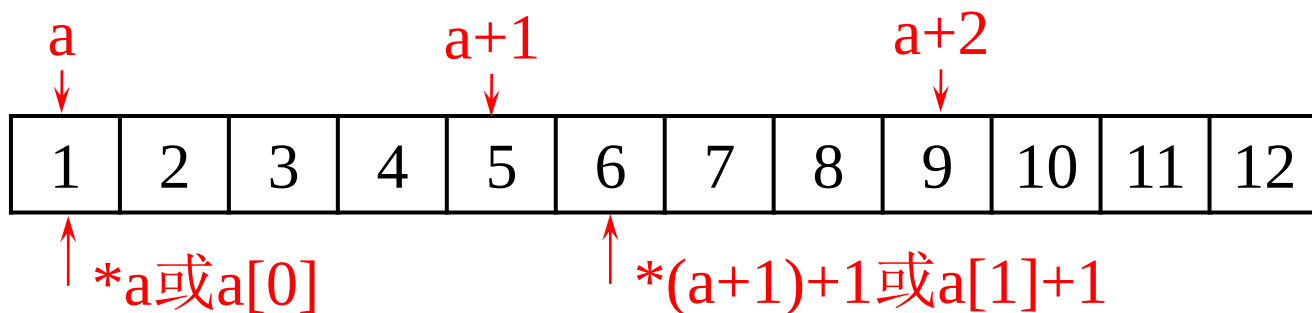
■ 第 1 行的首地址为 $a + 1$

➤ $a+1$ 、 $\&a[1][0]$ 、 $a[1]$ 、 $*(a+1)$

■ 第 2 行的首地址为 $a + 2$

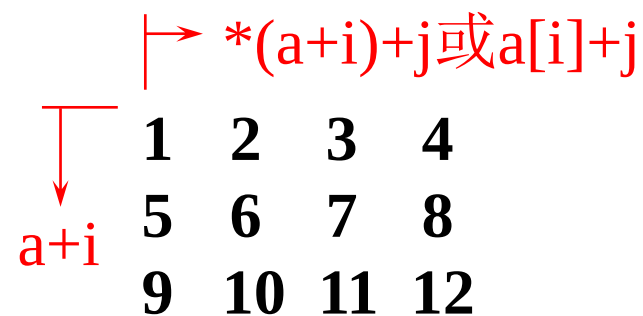
➤ $a+2$ 、 $\&a[2][0]$ 、 $a[2]$ 、 $*(a+2)$

```
int a[3][4]={ {1,2,3,4}, {5,6,7,8}, {9,10,11,12} };
```



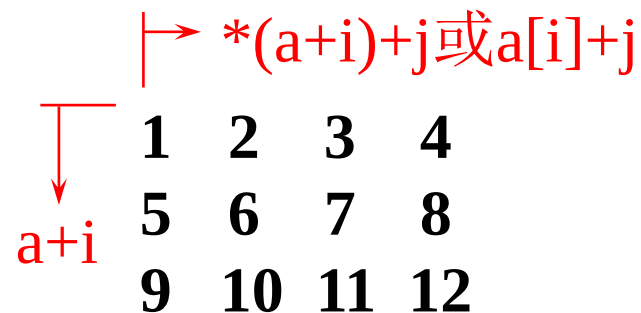
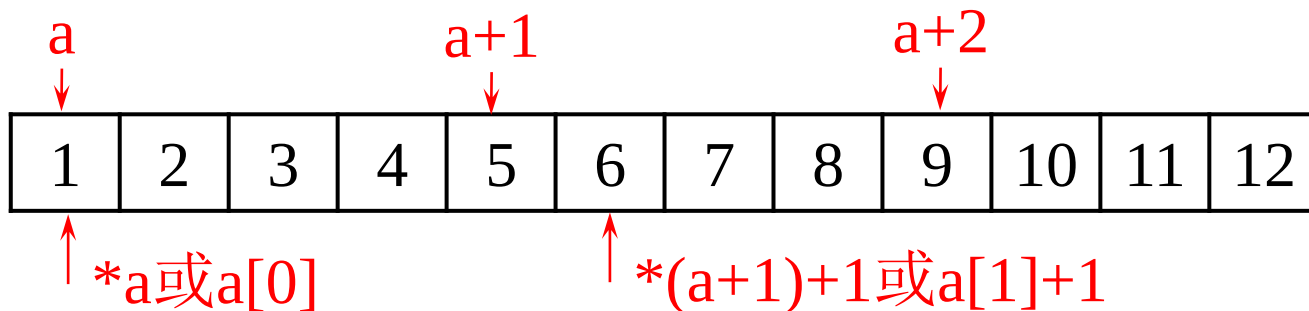
◆ 元素地址

■ $\&a[i][j]$ 、 $a[i]+j$ 、 $*(a+i)+j$





```
int a[3][4]={ {1,2,3,4}, {5,6,7,8}, {9,10,11,12}};
```



体会a与a[0]的异同?

```
cout << a << endl;  
cout << a[0] << endl;
```

```
cout << a+1 << endl;  
cout << a[0]+1 << endl;
```

a、a[0]都代表数组的首地址
a+1: 下一行首地址
a[0]+1: 下一个元素首地址

理解a与*a的异同?

```
cout << a << endl;  
cout << *a << endl;
```

```
cout << a+1 << endl;  
cout << *a+1 << endl;
```

a和*a地址的数值相同;
a+1和*a+1分别为下行首地址和下一个元素地址



◆ 引用二维数组元素的方法

■ 数组元素下标法

$a[i][j]$ 等价于:

$*(a[i]+j)$

$*(*(a+i)+j)$

$*(&a[0][0]+col*i+j)$

■ 指针访问二维数组

`int *p, a[3][4];`

`p = a[0];`

$a[i][j]$ 等价于 $*(p+4*i+j)$

数组元素在内存中连续存储，故可以利用 $p+n$ 逐一访问

◆ 给出以下程序的运行结果

```
int a[3][5]={1,2,3,4,5,6,7,8,9,10,11,12,13,14,15};
int *p;
for (p = a[0]; p < a[0] + 15; p++)
    cout << *p << " ";
p = a[0];
cout << endl;
```

D:\Edu-CPP\chap09\ex09-05.exe

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

若利用指针操作二维数组，必须让p指向a[0]（ $p=a[0]$ ），而不是a，为什么？

【1】 $p=a$: p指向二维数组（元素为一维数组的一维数组）， $p+1$ 指向数组下一行

【2】 $p=a[0]$: p指向元素为整数的一维数组（二维数组第0行）， $p+1$ 指向下一个整数， $p++$ 可以“越界”访问紧随第0行之后的下一行的元素，遍历整个二维数值

本章主要内容



9.1 指针与指针变量

9.2 指针运算

9.3 用指针处理数组

9.4 指针数组与数组指针



9.5 指针作为函数参数

9.6 指针型函数和指向函数的指针

9.7 动态内存分配

9.8 作业与实践



指针数组

◆ 指针数组的每个元素都是指针变量，并指向相同的数据类型

◆ 指针数组定义语法格式

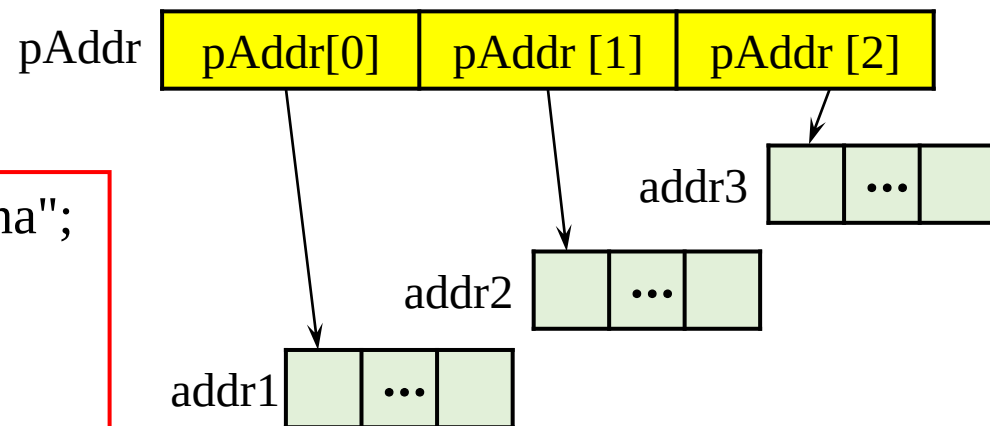
■ 数据类型 * 指针数组名 [常量表达式];

■ 例如: `int *p[4];`

可以写为: `int* p[4];`
由 `p[0]`、`p[1]`、`p[2]`、`p[3]`
四个指向整数的指针组成

```
char addr1[] = "485 Danxia Road, Hefei, Anhui, P. R. China";  
char addr2[] = "193 Tunxi Road";  
char addr3[] = "301 Xunhua Road";  
char *pAddr[3] = {addr1, addr2, addr3};  
for(int i = 0; i < 3; i++)  
    cout << pAddr[i] << endl;
```

指针数组实现二维数组的操作



D:\Edu-CPP\chap09\ex09-06.exe

```
485 Danxia Road, Hefei, Anhui, P. R. China  
193 Tunxi Road  
301 Xunhua Road
```



◆ 给出以下程序的运行结果

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    int a[3][5] = {1,2,3,4,5,6,7,8,9,10,11,12,13,14,15};
    int *p[3] = {a[0], a[1], a[2]};
    for(int i = 0; i < 3; i++)
    {
        for(int j = 0; j < 5; j++)
            cout << setw(5) << p[i][j];
        cout << endl;
    }

    return 0;
}
```

可以把二维数组看作
一种特殊的一维数组，
其元素又是一维数组

D:\Edu-CPP\chap09\ex09-07.exe

1	2	3	4	5
6	7	8	9	10
11	12	13	14	15



数组指针

◆ 数组指针：定义一个指针，而指针所指的对象是一个数组

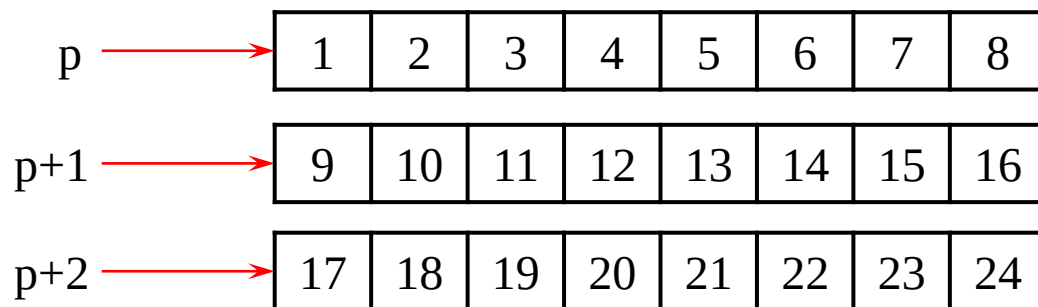
◆ 数组指针定义语法格式

■ 类型说明符 (*指针名)[长度];

■ 例如：int (*p)[8];

指向长度为8的整型数组

括号不可省略，表示p先与*结合，是指针变量，然后再与后面的[]结合，表示此指针指向数组；没有括号则为指针数组



指针p和整数n的算术运算 $p \pm n$ 后p指向的新地址是 $p \pm \text{sizeof}(p\text{的类型}) * n$

本章主要内容



9.1 指针与指针变量

9.2 指针运算

9.3 用指针处理数组

9.4 指针数组与数组指针

9.5 指针作为函数参数



9.6 指针型函数和指向函数的指针

9.7 动态内存分配

9.8 作业与实践



传引用与传地址的对比

◆ 传引用

```
void swap(int& p, int& q)
{
    int temp = p;
    p = q;
    q = temp;
}

int main()
{
    int x = 2, y = 3;
    cout << "before: x=" << x << ", y=" << y << endl;
    swap(x, y);
    cout << "after: x=" << x << ", y=" << y << endl;
    return 0;
}
```

◆ 传地址（传指针）

```
void swap(int *p, int *q)
{
    int temp = *p;
    *p = *q;
    *q = temp;
}

int main()
{
    int x = 2, y = 3;
    cout << "before: x=" << x << ", y=" << y << endl;
    swap(&x, &y);
    cout << "after: x=" << x << ", y=" << y << endl;
    return 0;
}
```

D:\Edu-CPP\chap09\ex09-08B.exe

before: x=2, y=3
after: x=3, y=2

形参是指针，实参一定是地址



示例：指针作为函数参数

◆ 写一个函数，输入浮点数，输出小数部分和整数部分；在主函数中读入三个浮点数，调用该函数将其整数和小数部分分别输出

■ 分析：如何让一个函数返回多个结果（小数部分和整数部分）？

➤ `void splitFloat(float fNum, int* pI, float* pF)`

➤ 当需要返回多个数值时，可定义指针变量作为形参

```
#include <iostream>
using namespace std;

void splitFloat(float fNum, int *pI, float *pF) {
    *pI = int(fNum);
    *pF = fNum - *pI;
}

int main() {
    float fNum, fPart;
    int iPart;
    cout << "请输入三个浮点数\n";
    for(int i = 0; i < 3; i++)
    {
        cin >> fNum;
        splitFloat(fNum, &iPart, &fPart);
        cout << fNum << "的整数部分=" << iPart << ", 小数部分=" << fPart << '\n';
    }
    return 0;
}
```

D:\Edu-CPP\chap09\ex09-09.exe

请输入三个浮点数

25.67

25.67的整数部分=25, 小数部分=0.67

3.14

3.14的整数部分=3, 小数部分=0.14

56.78

56.78的整数部分=56, 小数部分=0.779999



一级指针/一维数组作为函数参数

◆ 常量指针作为形参

- 当用指针作为形参时，实参与形参指向同一块内存，如果不希望实参在函数中被改变时，可定义常量指针作为形参

```
#include <iostream>
using namespace std;

void print(const int* p, int n)
{
    cout << "{";
    for(int i = 0; i < n; i++)
        cout << *(p+i) << ",";
    cout << "\b}";
}
```

```
int main( )
{
    const int N = 6;
    int array[N] = {1, 2, 3, 4, 5, 6};
    print(array, N);

    return 0;
}
```

形参是指针，实参可以是指针或数组
形参是数组，实参可以是指针或数组

D:\Edu-CPP\chap09\ex09-10.exe

{1,2,3,4,5,6}



```
{78, 5, 4, 23, 5, 56}
{4, 5, 5, 23, 56, 78}
```

◆ 指针常量作为形参

■ 当需要传递一组数给函数或返回多个值时，形参可以是指针（数组）。为了**防止指针在运算过程中指向不安全地方**，可以使用**指针常量**

```
#include <iostream>
```

```
using namespace std;
```

```
const int N = 6;
```

不允许修改指针指向的对象

```
void print(const int* p, int n) {
```

```
    cout << "{";
```

```
    for(int i = 0; i < n; i++)
```

```
        cout << *(p+i) << ",";
```

```
    cout << "\\b}\\n";
```

```
}
```

```
void swap(int *p, int *q) {
```

```
    int temp = *p;
```

```
    *p = *q;
```

```
    *q = temp;
```

```
}
```

```
void selectSort(int* const pArray, int n) {
```

```
    int index, temp;
```

```
    for(int loop = 0; loop < n-1; loop++) {
```

```
        index = loop;
```

```
        for(int i = loop + 1; i < n; i++)
```

```
            if(pArray[i] < pArray[index]) index = i;
```

```
        if(loop != index)
```

```
            swap(pArray + loop, pArray + index);
```

```
    }
```

```
}
```

```
int main() {
```

```
    int array[N] = {78, 5, 4, 23, 5, 56};
```

```
    print(array, N);    selectSort(array, N);    print(array, N);
```

```
    return 0; }
```

不允许修改指针

二级指针/二维数组作为函数参数



◆方法1：给定第2维长度的二维数组作为形参传递

```
void printArray1(int arr[][5], int row)
{
    for(int i = 0; i < row; i++)
    {
        for(int j = 0; j < 5; j++)
            cout << arr[i][j] << " ";
        cout << endl;
    }
}
```

◆方法2：使用给定第2维长度的数组指针作为形参传递

```
void printArray2(int (*arr)[5], int row)
{
    for (int i = 0; i < row; i++)
    {
        for (int j = 0; j < 5; j++)
            cout << arr[i][j] << " ";
        cout << endl;
    }
}
```



◆方法3：将二维数组指针转换成一维数组指针作为形参传递

```
void printArray3(int *arr, int row, int col)
{
    for (int i = 0; i < row; i++)
    {
        for (int j = 0; j < col; j++)
            cout << arr[i*col + j] <<" ";
            // *(arr + i * col + j);
        cout << endl;
    }
}
```

◆方法4：使用二级指针作为形参传递

```
void printArray4(int **arr, int row, int col)
{
    for(int i = 0; i < row; i++)
    {
        for(int j = 0; j < col; j++)
            cout << *((int*)arr + i*col + j) << " ";
        cout << endl;
    }
}
```



```
int main()
{
    int array[3][5] = {{11,12,13,14,15},
                      {21,22,23,24,25},
                      {31,32,33,34,35}};

    printArray1(array, 3);
    cout << endl;
    printArray2(array,3);
    cout << endl;
    printArray3((int *)array, 3, 5);
    cout << endl;
    printArray4((int **)array, 3, 5);
    return 0;
}
```

> D:\Edu-CPP\chap09\ex09-12.exe

```
11 12 13 14 15
21 22 23 24 25
31 32 33 34 35
```

```
11 12 13 14 15
21 22 23 24 25
31 32 33 34 35
```

```
11 12 13 14 15
21 22 23 24 25
31 32 33 34 35
```

```
11 12 13 14 15
21 22 23 24 25
31 32 33 34 35
```

本章主要内容



9.1 指针与指针变量

9.2 指针运算

9.3 用指针处理数组

9.4 指针数组与数组指针

9.5 指针作为函数参数

9.6 指针型函数和指向函数的指针



9.7 动态内存分配

9.8 作业与实践



指针类型函数

◆ 指针类型函数，即函数返回值是地址，例如：**int *f(int x, int y);**

```
int max (int x, int y)
{
    int z;
    if (x>y)
        z = x;
    else
        z=y;
    return z;
}
```

改写

```
int *max (int x, int y)
{
    int z;
    if (x>y)
        z = x;
    else
        z=y;
    return &z;
}
```

一定要返回合法地址，
不要将非静态局部地址用作函数返回

```
int z; // 全局变量
int *max(int x, int y)
{
    z = x>=y ? x : y ;
    return &z;
}
```

不提倡

```
int main()
{
    int a, b, *p;
    cin >> a >> b;
    p = max(a, b);
    cout << *p << "\n";

    return 0;
}
```

D:\Edu-CPP\chap09\ex09-13.exe

Enter two integer:20 30
30



◆ 返回传入数据的地址，避免使用全局变量

```
int *max (int x[ ], int len)
{
    int index = 0;
    for (int i = 1; i < len; i++)
        if (x[i] > x[index]) index = i;
    return &x[index];
}
```

```
int main( )
{
    int a[ ] = {1, 9, 7, 1, 8}, *p;
    p = max(a, 5);
    cout << *p << "\n";
    return 0;
}
```

D:\Edu-CPP\chap09\ex09-14.exe

9

```
#include <iostream>
using namespace std;

int* reverse(int* list, int size)
{
    for (int i = 0, j = size - 1; i < j; i++, j--)
    {
        int temp = list[j];
        list[j] = list[i];
        list[i] = temp;
    }

    return list;
}
```

```
int main()
{
    int list[] = {1, 2, 3, 4, 5};
    int *p = reverse(list, 5);
    for (int i = 0; i < 5; i++)
        cout << p[i] << " ";
    return 0;
}
```

D:\Edu-CPP\chap09\ex09-15.exe

5 4 3 2 1



函数指针

- ◆ 函数指针是指向函数的指针变量。每个函数都占用一段内存单元，有一个起始地址，函数指针指向**函数入口地址**，可以利用函数指针调用函数
- ◆ 定义格式： 函数返回值类型 (*指针变量)(函数参数列表);

```
#include <iostream>
using namespace std;
```

```
int max(int x, int y)
{
    return x > y ? x : y;
}
```

```
int main ()
{
```

```
    int (*p)(int, int);
```

```
    int a, b, c;
```

```
    p = max;
```

```
    cout << "输入两整数: ";
```

```
    cin >> a >> b;
```

```
    c = (*p)(a, b); // c = p(a, b); 低版本编译器不支持
```

```
    cout << a << "和" << b << "中最大数是" << c << "\n";
```

```
    return 0;
```

```
}
```

括号不可省略，表示p先与*结合，是指针变量，然后再与后面的()结合，表示此指针指向函数

```
D:\Edu-CPP\chap09\ex09-16.exe
```

```
输入两整数: 20 60
20和60中最大数是60
```



函数指针作为参数

引例

```
#include <iostream>
using namespace std;

int max(int x,int y)
{
    return x>y ? x : y;
}

int min(int x,int y)
{
    return x<y ? x : y;
}

int add(int x, int y)
{
    return x + y;
}
```

```
int main()
{
    int a, b;
    int (*fun)(int,int);
    cout << "输入两整数: ";
    cin >> a >> b;
    fun = max;
    cout << "max=" << fun(a, b);
    fun = min;
    cout << " \nmin=" << fun(a, b);
    fun = add;
    cout << " \nsum=" << fun(a, b);
    return 0;
}
```

```
int process(int x, int y, int (*fun)(int, int))
{
    return (*fun)(x, y);
}

int main()
{
    int a, b;
    cout << "输入两整数: ";
    cin >> a >> b;
    cout << "max=" << process(a, b, max);
    cout << " \nmin=" << process(a, b, min);
    cout << " \nsum=" << process(a, b, add);

    return 0;
}
```

D:\Edu-CPP\chap09\ex09-17.exe

```
输入两整数: 20 30
max=30
min=20
sum=50
```

本章主要内容



9.1 指针与指针变量

9.2 指针运算

9.3 用指针处理数组

9.4 指针数组与数组指针

9.5 指针作为函数参数

9.6 指针型函数和指向函数的指针

9.7 动态内存分配

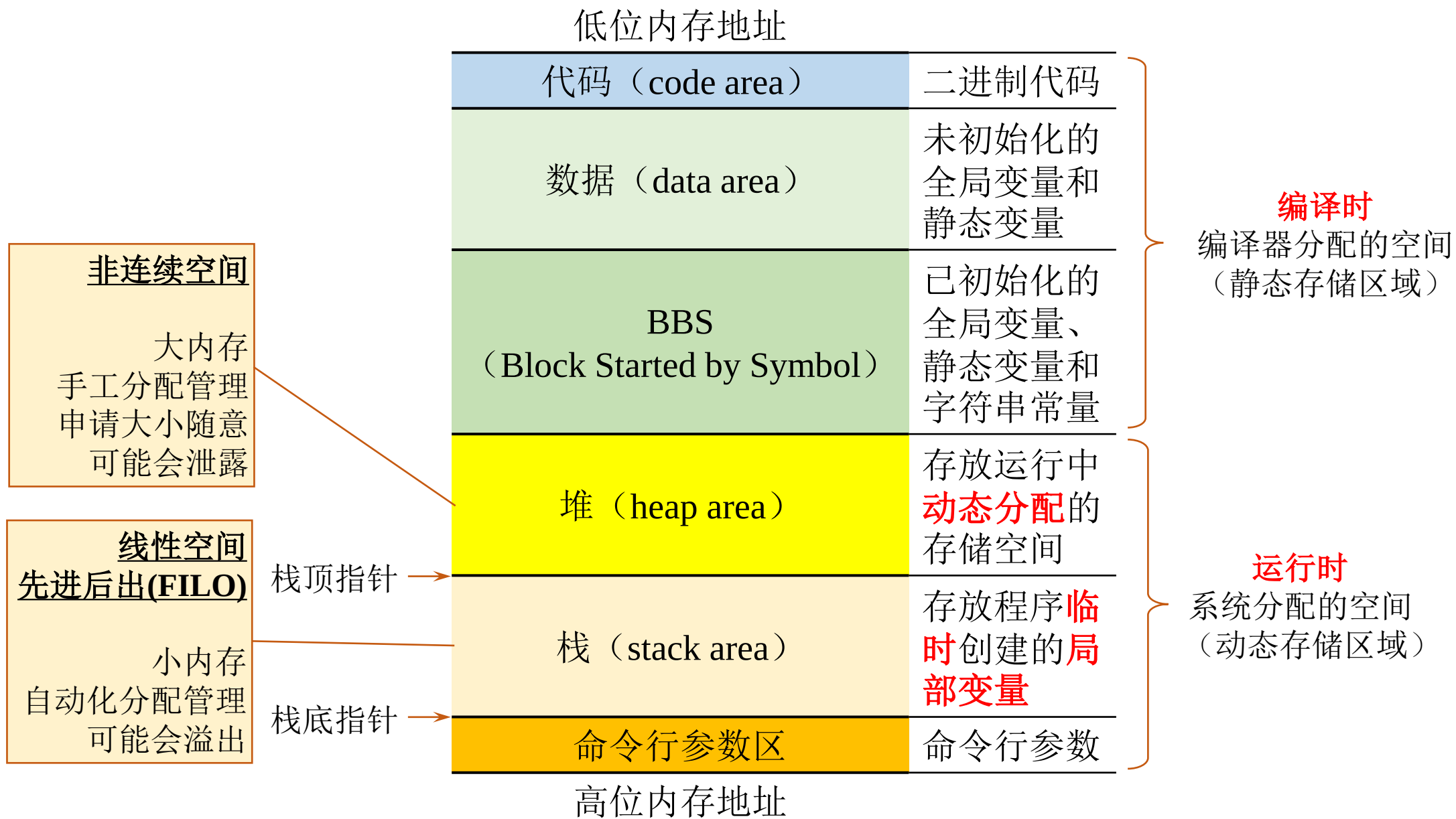


9.8 作业与实践

动态内存分配概述



- ◆ 动态内存分配是指在**程序执行过程中**动态地分配或回收存储空间
 - 与**静态内存分配**不同，动态内存分配不需要**预先**分配存储空间，而是根据实际需求**即时**分配，尤其是在处理**不确定数据量**时，能提供更灵活的内存管理
 - 动态存储分配包括栈式和堆两种分配方式。**栈式分配**通常用于存储**局部变量**，具有先进后出的特性；而**堆式分配**则用于存储程序运行过程中**动态申请**的内存
 - 在C语言中，动态内存分配主要通过**malloc、calloc和free等函数**实现；而在C++中，则使用**new和delete运算符**



利用new和delete运算符动态分配和释放内存



◆ new

- 使用格式：数据类型 指针 = new 数据类型 （例如：int *p = new int;）
- 功能：在程序执行期间，申请某种数据类型的变量或对象空间，并用指针指向
- 结果值
 - 成功——指针指向新分配的内存
 - 失败——抛出异常

◆ new的3种使用方式

- 给单个对象申请分配内存

```
int *ip = NULL;  
ip = new int;
```

```
delete ip;
```

- 给单个对象申请分配内存的同时初始化该对象

```
int *ip = NULL;  
ip = new int(68);
```

```
delete ip;
```

- 同时给多个对象申请分配内存

```
int *ip = NULL;  
ip = new int [5];
```

```
delete [ ] ip;
```



◆ 注意事项

- 用new申请分配的内存空间，必须用delete释放
- 对一个已分配内存的指针，只能用delete释放一次
- delete作用的指针对象必须是由new分配内存空间的首地址
- 用new运算符为多个对象申请分配内存空间时，不能提供初始化

malloc、calloc等函数分配的内存，必须用free函数释放

教材第371页程序清单11-9存在内存泄漏

创建一维动态数组



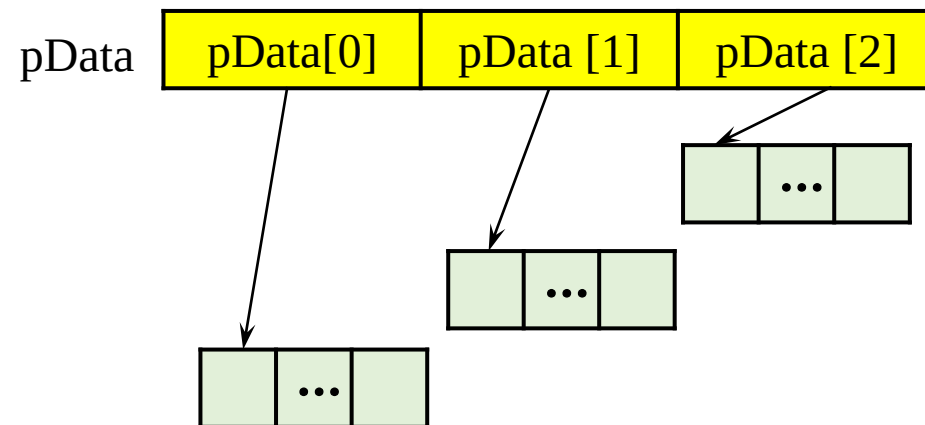
```
int count = 10;  
int *p = NULL;  
p = new int[count]; // 申请可以存放10个整数的内存空间  
for(int i = 0; i < count; i++)  
    cin >> p[i]; // cin >> *(p + i);  
.....  
delete[ ] p; // 不再使用时一定要释放，否则内存泄漏
```

创建二维动态数组



```
int n = 5, m = 6 ;  
long **pData = new long*[n]; // 申请可以存放n个指针的数组  
for(int j = 0; j < n; j++)  
    pData[j] = new long[m]; // 申请可以存放m个长整数的数组  
  
for (int i = 0; i < n; i++)  
    for (int j = 0; j < i+1; j++) cin >> pData[i][j];  
.....  
for(int i = 0; i < n; i++)  
    delete[ ] pData[i]; //用完一定要释放  
delete[ ] pData;
```

注意释放内存的先后顺序，避免访问无效内存





(回顾) 示例3: 矩阵转置

◆ 已知行列的矩阵转置: $a = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$ 转换成 $b = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    int a[2][3]={{1, 2, 3}, {4, 5, 6}};
    int b[3][2], i, j;
    cout << "array a: \n";
    for(i = 0; i <= 1; i++)
    {
        for(j = 0; j <= 2; j++)
        {
            cout << setw(5) << a[i][j];
            b[j][i] = a[i][j];
        }
        cout<<"\n";
    }
}
```

```
cout << "\narray b: \n";
for(i = 0; i <= 2; i++)
{
    for(j = 0; j <= 1; j++)
        cout<<setw(5)<<b[i][j];
    cout<<"\n";
}
return 0;
}
```

```
D:\Edu-CPP\chap08\ex08-05.exe

array a:
    1    2    3
    4    5    6

array b:
    1    4
    2    5
    3    6
```

动态分配内存示例：任意行列的矩阵转置



```
#include <iostream>
#include<iomanip>
using namespace std;

void inputArray(int **arr, int row, int col)
{
    for(int i = 0; i < row; i++)
        for(int j = 0; j < col; j++)
            cin >> *((int*)arr + i * col + j);
}

void outputArray(int **arr, int row, int col)
{
    for(int i = 0; i < row; i++)
    {
        for(int j = 0; j < col; j++)
            cout << setw(5) << *((int*)arr + i * col + j);
        cout << "\n";
    }
}
```

```
int** matrixTrans(int **arr, int row, int col)
{
    int **Trans = new int*[col];
    for(int j = 0; j < col; j++)
        Trans[j] = new int[row];
    for(int i = 0; i < row; i++)
    {
        for(int j = 0; j < col; j++)
            *((int*)Trans + j * row + i) = *((int*)arr + i * col + j);
    }
    return Trans;
}
```



```
int main()
{
    int n = 2, m = 5, **p = NULL;
    int **pData = new int*[n];
    for(int j = 0; j < n; j++)
        pData[j] = new int[m];
    inputArray(pData, n, m);
    cout << "转置前: \n";
    outputArray(pData, n, m);
    p = matrixTrans(pData, n, m);
    cout << "转置后: \n";
    outputArray(p, m, n);
    for(int i = 0; i < n; i++)
        delete[ ] pData[i];
    delete[ ] pData;
    for(int i = 0; i < m; i++)
        delete[ ] p[i];
    delete[ ] p;
    return 0;
}
```

注意释放内存的先后顺序，避免访问无效内存

> D:\Edu-CPP\chap09\ex09-18.exe

```
11 12 13 14 15
21 22 23 24 25
转置前:
    11    12    13    14    15
    21    22    23    24    25
转置后:
    11    21
    12    22
    13    23
    14    24
    15    25
```

本章主要内容



9.1 指针与指针变量

9.2 指针运算

9.3 用指针处理数组

9.4 指针数组与数组指针

9.5 指针作为函数参数

9.6 指针型函数和指向函数的指针

9.7 动态内存分配

9.8 作业与实践



第9章作业



- (1)编写程序使用动态数组，先读入数组长度，在读入数组中每个数，输出其中不同的数（即若一个数出现多次，只输出一次）。提示：读入一个数，若未出现过，将其存入数组，否则底漆，输入完毕即输出
- (2)编写函数`int* count(const string& s)`统计字符串中每个数字出现的次数，该函数返回值是一个有10个元素的数组，每个元素表示一个数字出现的次数。编写测试程序调用该函数，输出统计结果
- (3)编写三个函数分别实现冒泡排序、选择排序和归并排序，要求使用动态数组
- (4)编写函数`void min_max_element(int *array, int n, int *minpos, int *maxpos)` 获取整型数组中最小和最大元素下标，如果有多个相同极值元素，返回最小下标，例如{2, 4, 4, 5, 7, 8, 8, 2}结果为最小值下标=0, 最大值下标=5